Ticket Reservation System

Final project report · Java 17 · Six integrated design patterns

Team

Natalyev · eslishnah · MangoDeath

Repository

github.com/Natalyev/sdp-final-

Deliverables

Source code, editable UML, JUnit tests, terminal run evidence, and this report.

1. Abstract

Ticket Reservation System is an in-memory Java application for concert and flight bookings. Users register events, book numbered seats, confirm or cancel reservations, change prices, and switch seating policies. Views receive seat and price notifications. The application integrates Factory Method, Strategy, Observer, Builder, State, and Facade in one workflow.

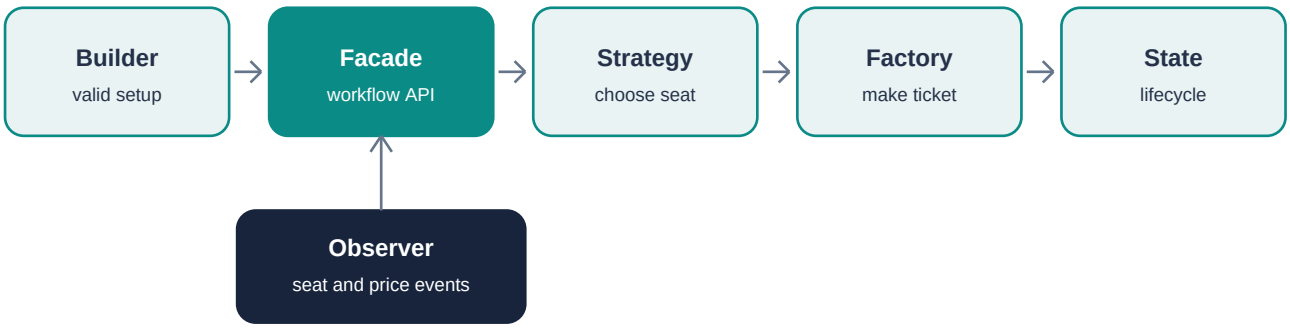
2. Requirements and scenario

Each event has its own ticket category, price, seating policy, and available seats. Booking chooses one available seat and creates the correct ticket. A booking starts pending. Confirmation and cancellation follow controlled transitions. Cancellation returns a seat. Invalid IDs, duplicate events, exhausted inventories, and illegal transitions are rejected.

In the demo, a customer books concert seat 1 using the front-row policy. The reservation is confirmed. The policy switches to back-row, so the next booking takes seat 3. Cancellation returns seat 3. A separate flight event then creates a flight ticket.

3. Architecture

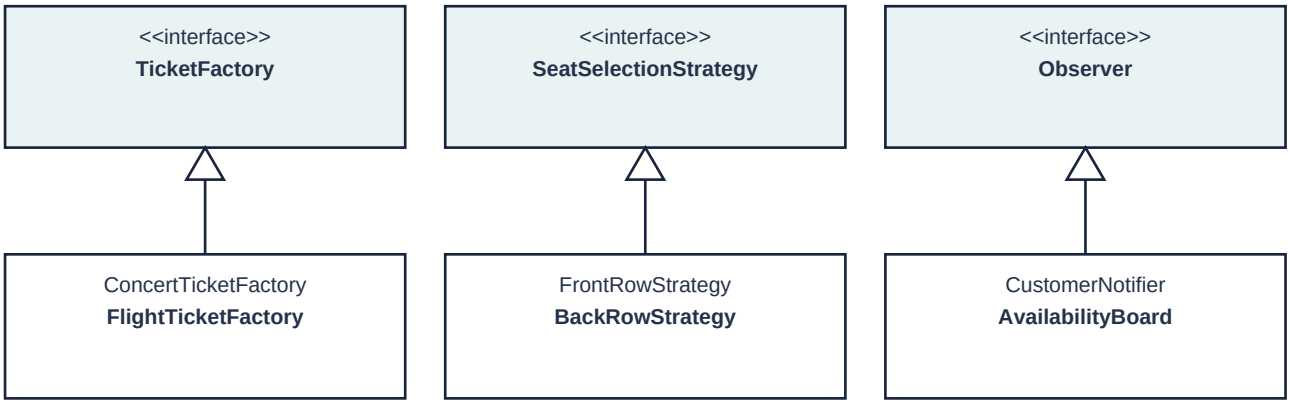
The Facade owns one service and inventory per event. Builder validates registration data. The service uses Strategy to select a seat and Factory Method to create its ticket. A reservation session delegates transitions to State objects. Observer distributes seat and price changes to subscribed views.



Editable UML class diagram: `docs/class-diagram.puml`.

4. UML class diagram

The following class relationships show the Factory Method, Strategy, and Observer interfaces with their concrete implementations. The editable PlantUML file contains the complete diagram, including Builder, State, Facade, and their associations.



Open triangles show implementation of the interface above each pair.

5. Pattern roles and integration

Pattern	Participants	Purpose
Factory Method	TicketFactory; concert and flight factories	The service creates a concrete ticket through a factory interface.
Strategy	SeatSelectionStrategy; front/back strategies	The inventory delegates seat choice; the facade can replace the policy at runtime.
Observer	TicketEventPublisher; notifier; board	Subscribed views receive seat-count or price events without service-to-view coupling.
Builder	EventConfiguration.Builder	A complete configuration is validated before an event is registered.
State	PendingState; ConfirmedState; CancelledState	The current state decides whether confirmation or cancellation is allowed.
Facade	TicketReservationSystem	A single API coordinates event setup, booking, lifecycle, pricing, and subscriptions.

6. Java code example

The client assembles a complete event, then books and confirms through the Facade:

```
system.register(new EventConfiguration.Builder("SHOW")
    .factory(new ConcertTicketFactory())
    .strategy(new FrontRowStrategy())
    .seats(Set.of(1, 2, 3))
    .price(new BigDecimal("50.00")).build());
Reservation booking = system.book("SHOW", "CUSTOMER-1");
system.confirm(booking.id());
```

7. Design decisions

- Separate inventories prevent concert and flight seats from interfering.
- Immutable reservation snapshots keep lifecycle changes inside the facade, where seat release also occurs.
- The inventory removes a chosen seat atomically and rejects an unavailable strategy result.
- If ticket creation fails, the selected seat is returned before the error is propagated.
- Synchronous Observer delivery and CopyOnWriteArrayList allow subscription changes during notification.

8. Verification

18 tests passed · 0 failures · 0 errors. Ten tests cover the initial patterns and service integration. Eight workflow tests cover all-pattern integration, Builder validation, state transitions, seat reuse, independent events, interactive commands, rollback, and per-event availability counts.

Actual terminal run evidence is provided in the three text files under `evidence/`.

```
Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

9. How to run

```
mvn clean test
mvn exec:java
mvn package
java -cp target/classes tickets.Main --interactive
```

Interactive commands: `help`, `events`, `book EVENT CUSTOMER`, `confirm ID`, `cancel ID`, `show ID`, `price EVENT AMOUNT`, `strategy EVENT front|back`, and `quit`.

10. Limitations and extensions

The application is in-memory: data is lost when it exits. It does not process payments, authenticate accounts, display a visual seat map, or book multiple seats in one order. A database and graphical interface are natural future extensions; neither is needed to demonstrate the integrated patterns.

11. Execution evidence

The following excerpts come from actual runs of the current code. Full transcripts are included in the `evidence/` folder.

Demo run

```
Booked concert seat: 1 (PENDING)
Confirmed reservation: R-1 (CONFIRMED)
After strategy switch, seat: 3
Cancelled reservation; concert seats available: 2
Flight ticket: FlightTicket[eventId=FLIGHT-1, seat=11, price=200.00]
```

Interactive run

```
CONCERT-1: 3 seats available
EVENT SEAT_AVAILABILITY_CHANGED CONCERT-1 seats=2 price=50.00
R-1 PENDING ConcertTicket[eventId=CONCERT-1, seat=1, price=50.00]
R-1 CONFIRMED ConcertTicket[eventId=CONCERT-1, seat=1, price=50.00]
R-1 CANCELLED ConcertTicket[eventId=CONCERT-1, seat=1, price=50.00]
CONCERT-1: 3 seats available
```

12. Conclusion

The prototype demonstrates the requested Ticket Reservation System topic. Six patterns cooperate in event setup, seat selection, ticket creation, reservation transitions, and notifications. The Maven build succeeds and all 18 tests pass.

13. Further work

Persistent storage, payment processing, multi-seat orders, authentication, and a graphical seat map would extend the prototype for broader use.

14. Team and repository

The repository records work under the GitHub handles Natalyev, eslishnah, and MangoDeath. Its commit history is the source for exact file-level attribution. Full names can be provided in the course submission form if required.

Repository: https://github.com/Natalyev/sdp-final-.